

团队作业 2——设计文档

抗日战争胜利 80 周年纪念网站（Yii2 Advanced / 前后台分离）

0. 写在前面：项目定位与范围

本项目以“纪念性专题站”的方式组织内容：既要让普通用户**快速进入主题、沉浸式浏览**，也要保证站点内容**可维护、可扩展、可治理**。因此系统采用 Yii2 Advanced 模板的前后台分离结构：前台负责展示与交互，后台负责录入与管理，公共层沉淀模型与配置，形成“后台可配置、前台动态渲染”的工作流。

整体内容围绕三类信息展开：

一类是“历史叙事”（专题首页入口、时间轴大事件）；一类是“空间与数据可视化”（战役地标地图、统计数据图表）；一类是“人物与文化资源”（人物专栏、影视资源、留言互动）。项目在设计上尽量让每一类信息形成**最小闭环**：后台能录入、前台能展示、数据能追溯、内容能下架而不丢历史。

1. 系统架构设计

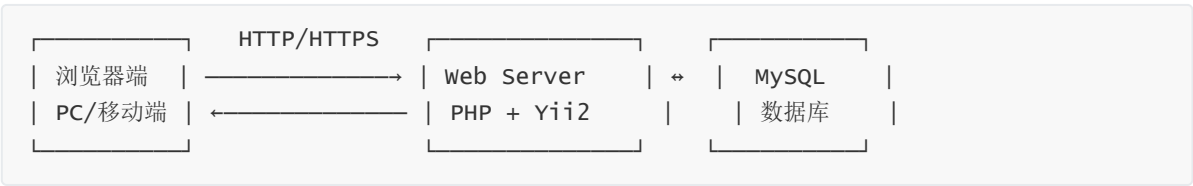
1) 总体架构：MVC + 前后台分离

系统基于 Yii 框架的 MVC 结构组织业务，同时采用 Yii2 Advanced 的目录约定实现前后台分离。它的好处很直接：展示端可以更大胆做视觉与交互，管理端可以更稳定地做增删改查；二者共享同一套数据模型与数据库配置，不会出现“写两套模型、维护两份逻辑”的重复劳动。



2) 部署架构

系统采用典型的 Web 部署方式：浏览器发起请求，Web Server（Apache/Nginx）转交 PHP 应用（Yii），Yii 访问 MySQL 并渲染页面/返回 JSON。



3) 目录结构

文档中涉及到的文件位置，默认遵循 Yii2 Advanced 组织方式：

- `frontend/controllers/*`：前台控制器（首页、时间轴、战役地图、人物、影视等）
- `frontend/views/*`：前台页面视图
- `frontend/web/*`：静态资源（css/js/geojson/uploads）
- `backend/controllers/*`：后台控制器（人物、影视资源、首页入口、战役等）
- `backend/views/*`：后台 CRUD 视图与表单
- `common/models/*`：共享数据模型（Figure、MediaResource、TimelineEvent、HomepageFeature、Battle 等）
- `console/migrations/*`：迁移文件（建表、字段变更）

2. 数据库与数据模型设计

1) 总体思路：内容站的数据要“能写、能读、能下架、能追溯”

纪念站最怕两件事：一是后台录入很费劲、字段过碎导致编辑体验差；二是内容上线后无法治理，想撤下只能硬删，最后引用关系断裂。为此我们在表设计上做了几条统一约定：

- **展示类内容尽量“少字段但不空”**：把信息密度放进叙事文本（例如人物用 biography + achievements，而不是几十个零碎字段）。
- **可治理优先**：对需要下架的内容提供 `status` 或 `deleted_at`（软删）策略。
- **封面与资源链接分离**：封面是给列表渲染的，资源链接是给详情跳转的，职责不同，混在一起最容易“裂图”。

2) 核心实体与关系

本项目的“内容核心”可以理解为五组实体：

- 首页入口（HomepageFeature）：负责把站点内容“编排成入口”
- 时间轴事件（TimelineEvent）：负责把历史叙事串起来
- 战役（Battle）：负责地图可视化与战役详情（经纬度是关键）
- 人物（Figure）：负责英雄人物故事的卡片化与长文阅读
- 影视资源（MediaResource）：负责文化资源入口，并可挂载到人物等实体上

其中 MediaResource 采用“多态关联”设计（`linkable_type/linkable_id`），它不是只能连人物，未来也可以连时间轴事件、战役页面，作为一种通用“资源挂载器”。

3) 数据表设计（重点表详细说明）

时间轴事件表 `timeline_event`

时间轴页面要求“卡片摘要 + 弹窗长文”的分层阅读，因此事件表至少需要：日期、标题、摘要、封面、重要性、排序与状态。若未来把长文也入库，可追加 `detail` 字段。

字段	含义
id	主键
event_date	事件日期（用于年份轴与排序）
title	标题
description	摘要（卡片上显示的短文）
cover_image_url	封面图（卡片主视觉）
importance	重要性等级（用于标签语义化）
sort_order	运营排序（可选）
status	是否展示
created_at/updated_at	时间戳

人物表 figure

人物页最怕“填表式传记”，用户读起来碎，编辑写起来累。所以人物表只保留必要字段，把人物信息浓缩为两段：简介回答“你是谁”，功绩回答“你做了什么”。这不是减少内容，而是把内容组织得更像一篇可阅读的短文。

字段	含义
id	主键
name	姓名（必填）
biography	人物简介（叙事文本）
achievements	主要功绩/事迹（叙事文本）
cover_image_url	封面/头像图
created_at/updated_at	自动维护时间戳
deleted_at	软删时间（撤下但保留记录）

影视资源表 media_resource

影视资源不只是一条“影片信息”，更像一个可跳转的入口：它可以指向播放页、专题页、下载页，也可以是音频、图片或剪辑合集。因此我们用 type 做整数枚举，既便于过滤，也便于后期扩展。

字段	含义
id	主键
title	标题（必填）
type	类型枚举（电影/纪录片/电视剧/图文等）
description	简介

字段	含义
path	封面图路径/直链（优先用于列表渲染）
url	资源链接（用于详情页按钮跳转）
linkable_type	关联模型类名（如 Figure）
linkable_id	关联实体 ID
created_at/updated_at	时间戳

首页入口表 homepage_feature

首页不是“写死的一页”，而是一种编排：哪些内容在首页被看见、以什么顺序被看见、点进去到哪里。
homepage_feature 的意义就是把“首页入口”变成后台可配置的内容，减少改一张首页就要改代码的频率。

字段	含义
title/subtitle/description	标题、副标题与描述
image_url	图片
link_url	跳转链接
display_order	展示顺序
is_active	是否启用
created_at/updated_at	时间戳

战役表 battle

抗战地标模块选择复用战役表，而不是再造一张“地标表”，是为了保证“地图上的点”与“战役详情的正文”来自同一份数据，避免后期出现地图点改了、详情页没改的割裂。

战役表中最关键的是经纬度字段：main_latitude/main_longitude。只要它们有效，地图就能稳定渲染；其它字段决定详情页的信息密度（地点、时间、描述、结果、伤亡等）。同时，战役内容本身也属于“可维护、可更新”的叙事资产，因此表内保留 created_at/updated_at 便于审计与追溯。

字段设计（示例）：

字段	含义
id	主键
name	战役名称
start_date / end_date	起止日期（用于时间线排序/筛选）
main_location	主要地点（对用户友好展示）
main_latitude / main_longitude	地图点坐标（地图渲染关键字段）
description	战役简介/叙事正文（详情页主内容）

字段	含义
result	战役结果/影响摘要
casualties_china	中方伤亡（文本型，允许“约xx万人”等非结构表达）
casualties_japan	日方伤亡（同上）
created_at/updated_at	时间戳

留言表 `guestbook_message`

留言模块是站点“温度”的来源，但也天然带来治理压力：一方面要让用户愿意表达，另一方面必须确保内容可控、可审核、可回溯。因此留言表采用“文本内容 + 审核开关 + 时间戳”的朴素结构，避免字段过多让后台管理变得笨重，同时为后续扩展（例如增加 IP、敏感词命中标记、管理员回复等）留出空间。

在实际实现中，留言时间戳使用 `INT(11)`（Unix 时间），这一点与部分表使用 `created_at/updated_at` 的 `datetime/timestamp` 不同，但在 Yii2 中并不冲突：只要模型层统一维护时间戳，展示层按格式化输出即可。出于索引与排序性能考虑，`created_at` 建索引也更利于列表分页与按时间倒序展示。

字段设计：

字段名	类型	约束	说明
id	INT	PRIMARY KEY, AUTO_INCREMENT	主键ID
nickname	VARCHAR(100)	NOT NULL, DEFAULT ''	昵称
content	TEXT	NOT NULL	留言内容
is_approved	TINYINT(1)	NOT NULL, DEFAULT 0	审核状态：0-待审核，1-已通过
created_at	INT(11)	NOT NULL, DEFAULT 0	创建时间戳
updated_at	INT(11)	NOT NULL, DEFAULT 0	更新时间戳

建表 SQL：

```
CREATE TABLE `guestbook_message` (
  `id` INT NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `nickname` VARCHAR(100) NOT NULL DEFAULT '' COMMENT '昵称',
  `content` TEXT NOT NULL COMMENT '留言内容',
  `is_approved` TINYINT(1) NOT NULL DEFAULT '0' COMMENT '是否审核通过：0-待审核，1-已通过',
  `created_at` INT(11) NOT NULL DEFAULT '0' COMMENT '创建时间戳',
  `updated_at` INT(11) NOT NULL DEFAULT '0' COMMENT '更新时间戳',
  PRIMARY KEY (`id`),
  KEY `idx_is_approved` (`is_approved`),
  KEY `idx_created_at` (`created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='留言表';
```

重要纪念会议表 `important_meeting`

在纪念站里，除了时间轴与战役/人物这种“历史内容”，还会有一类更偏“现实纪念行动”的内容：**纪念大会、学术研讨会、主题展演、纪念节点活动**等。这些内容的共同特征是：

- 更像一个“入口”而不是纯文本：通常带封面、摘要、地点、日期，并可能跳转到权威外链或站内详情页；
- 需要可运营：要能控制展示与排序（`is_active`、`display_order`）；
- 需要可扩展：会议下往往还有“亮点/议题/议程摘要”等结构化内容，因此该表会和下方 `important_meeting_highlight` 形成 1 对多关系。

字段	含义
<code>id</code>	主键
<code>title</code>	会议标题（必填，用于列表与详情页主标题）
<code>meeting_date</code>	会议日期（用于时间排序与归档）
<code>location</code>	地点（展示信息，便于用户快速建立场景感）
<code>cover_image</code>	封面图（列表卡片主视觉）
<code>link_url</code>	外链/详情地址（可指向权威报道或站内详情路由）
<code>summary</code>	摘要（列表/详情页导语，可多行文本）
<code>display_order</code>	运营排序（同日期内也能人工调整展示顺序）
<code>is_active</code>	是否展示（下架不删数据，保证可追溯）
<code>created_at/updated_at</code>	时间戳（Unix 时间，便于统一维护与排序）

会议亮点表 `important_meeting_highlight`

一场大会/活动的详情页，最怕变成一整段难读的大段文字；但如果把内容拆得太碎（几十个字段）又会让后台录入痛苦。`important_meeting_highlight` 采用折中策略：**会议本体在 `important_meeting` 里，亮点按条目拆分**，每条亮点有标题与补充说明，并支持手工排序。

这个设计的好处是：前台详情页可以用“亮点列表/议题卡片”的方式组织阅读；后台也能像写新闻摘要一样逐条录入，不需要理解复杂字段结构。

字段	含义
<code>id</code>	主键
<code>meeting_id</code>	所属会议 ID（外键，指向 <code>important_meeting.id</code> ）
<code>title</code>	亮点/议题标题（必填）
<code>description</code>	补充说明（可为空，写细节或背景）

字段	含义
<code>display_order</code>	显示顺序（控制亮点展示先后）
<code>created_at/updated_at</code>	时间戳（Unix 时间）

用户表 `user`

虽然本项目是内容型纪念站，但只要存在后台录入与审核，就必须有一套“可登录、可追溯、可禁用”的账号体系。Yii2 Advanced 默认提供 `user` 表用于身份认证与会话保持，本项目沿用该设计。

字段	含义
<code>id</code>	主键
<code>username</code>	用户名（登录用，通常唯一）
<code>auth_key</code>	记住登录状态用的随机串（Cookie/自动登录校验）
<code>password_hash</code>	密码哈希（bcrypt 等算法产物，不可逆）
<code>password_reset_token</code>	找回密码 token（可为空）
<code>email</code>	邮箱（可用于通知/验证，通常唯一）
<code>status</code>	状态（常见：10=启用，0=禁用）
<code>created_at/updated_at</code>	时间戳（Unix 时间）
<code>verification_token</code>	邮箱验证 token（可为空）

抗战数据集目录表 `war_dataset`

`war_dataset` 这张表就是把“CSV 文件清单”变成后台可配置的数据目录：**前台读这个目录决定显示哪些数据集、按什么顺序显示、点进去加载哪个 CSV 文件**。这样做的好处是：

- 不用把每个 CSV 拆成多张明细表（成本很高，字段也不稳定）；
- 但又避免“写死文件名/写死页面入口”，让数据模块能运营（上架/下架/排序/改描述）；
- 后续如果某个 CSV 要升级成真正的数据库明细表，也能保留 `key` 作为稳定标识，平滑迁移。

字段	含义
<code>id</code>	主键
<code>key</code>	数据集唯一标识（程序内部引用用，建议唯一）
<code>name</code>	数据集显示名称（列表展示用）
<code>source_file</code>	对应 CSV 文件名/路径（前台读取数据用）
<code>description</code>	数据集说明（可选，用于解释来源/字段含义）
<code>display_order</code>	展示顺序（运营排序）
<code>is_active</code>	是否启用（下架不删，便于治理与回溯）

字段	含义
<code>created_at/updated_at</code>	时间戳（Unix 时间）

抗战数据明细表 `war_record`

`war_dataset` 解决的是“入口与治理”：哪些数据集要展示、对应哪个 CSV/来源文件、怎么排序。
但如果数据仍然只存在 CSV 里，前台每次都读文件、解析字段、处理脏数据；一旦后续想做“筛选/搜索/统计汇总/对比图表”，文件模式就会越来越吃力。

因此我们引入 `war_record`，把“每一行数据”抽象成**统一结构的记录**，用 JSON 存储不同数据集的异构字段：

- 表层结构保持稳定（`dataset_id + row_index + data_json`）；
- 数据字段可以随数据集变化（比如有的有 `period_start`，有的只有 `year`），避免为了每个 CSV 单独建表、单独写字段；
- 后续可直接用 SQL 做分页、筛选（按 `dataset_id`）、以及导出给前端 ECharts 使用。

也就是说：`war_dataset` 是“目录”，`war_record` 是“目录下的内容”。两者形成 1 对多关系：一个数据集对应多条记录。

字段	含义
<code>id</code>	主键
<code>dataset_id</code>	所属数据集（外键，指向 <code>war_dataset.id</code> ）
<code>row_index</code>	原始数据行号/序号（便于追溯 CSV 行、也便于稳定排序）
<code>data_json</code>	数据内容（JSON 字符串，容纳不同数据集的异构字段）
<code>created_at/updated_at</code>	时间戳（Unix 时间，用于治理：知道何时导入/更新、便于按时间追溯）

3. 模块设计与实现（前后端）

1) 专题首页模块

首页承担的不是“堆信息”，而是把用户引入叙事：先看到主题，再看到四五个“值得点进去”的入口。实现上，后台维护 `homepage_feature`，前台只负责取已启用内容并按顺序展示。纪念站里除了时间轴与战役/人物这种“历史叙事”，还会有一类更偏“现实纪念行动”的内容：纪念大会、学术研讨会、主题展演、纪念节点活动等。它们更像一个“入口”，而不是纯文本：用户关心的是**什么时候、在哪儿、有什么主题、点进去看什么**。因此我们把会议本体放在 `important_meeting`，把会议亮点/议题放在 `important_meeting_highlight`，形成 1 对多关系。

数据流很清晰：

后台管理员在 CRUD 中维护入口 → 前台 `SiteController::actionIndex()` 查询 `is_active=1` 并按 `display_order` 排序 → 视图将第一条作为主卡片，其余作为速览列表。为了保证部署到子目录时链接与图片不出错，前台对 `image_url/link_url` 做统一 `buildurl` 处理，把相对路径补全成可访问地址。

这种“入口表 + 前端渲染”的结构，最大的价值是：**首页会变得可运营**。因此我们不需要每次调整首页都改 PHP 文件，只要改后台配置即可。

2) 时间轴模块

时间轴页面的目标是让用户愿意停留。相比传统“竖向列表”，胶片式横向画廊更贴合“叙事观影”的心理节奏：用户不是在查资料，而是在沿着时间向前走。

页面交互分三层：

第一层是顶部年份轴（chips），帮助快速定位；第二层是横向卡片画廊，用 `scroll-snap` 保证滚动停在合适位置；第三层是详情弹窗，把长文放在弹窗里，既保持列表密度，也让阅读更沉浸。

数据结构上，我们把事件对象统一成前端可直接用的 `TIMELINE_EVENTS`（由后端渲染到 `window`），包含 `id/date/year/title/summary/poster/importance`。至于长文 `detail`，短期用本地 `detailMap` 提升表现力；长期建议在 `timeline_event` 增加 `detail` 字段，把长文也纳入内容治理体系，这样“详情文本”才能被后台编辑、审核、追溯。

重要性标签不直接显示 1/2/3，而用语义化表达（关键转折/重要节点/历史片段），是为了避免页面像“系统字段展示”，让用户读到的是叙事，而不是数据库。

3) 战役地标模块

地图页的设计核心是“空间索引”：用户可以从地理视角理解战役分布，悬停看到信息摘要，点击进入详情页完成阅读闭环。这里最关键的工程选择是：**ECharts 与中国地图 GeoJSON 本地化**。这意味着即使实验环境没有外网、没有 CDN，地图功能仍然可用，演示不会翻车。

前端实现逻辑是：

`BattleController::actionIndex()` 查询经纬度有效的战役数据 → 视图加载本地 `echarts.min.js` 和 `china.json` → 注册地图并用 `effectScatter` 标点 → `tooltip` 展示名称、时间与简介，且限制宽度自动换行以保证可读性。

战役详情页使用 `DetailView` 做字段展示，并将字段标签本地化为中文，让页面看起来更像“纪念站”，而不是“后台管理系统的前台复制品”。

4) 人物专栏模块

人物模块分前后台两部分：后台是内容录入与治理，前台是卡片化展示与详情阅读。

后台人物管理采用标准 CRUD，但删除采用软删：`actionDelete()` 写入 `deleted_at`，并在查询时默认过滤 `deleted_at IS NULL`。这样做的意义在纪念站里尤其明确：人物内容被撤下，往往不是“错误数据要抹掉”，而是“暂时不展示或待修订”。软删保留历史，也避免未来人物与资源、引用关系断裂。

前台人物列表用 `ActiveDataProvider` 分页（例如每页 12 条），卡片网格统一尺寸；点击进入详情页后，用 `biography + achievements` 形成一条完整叙事线，让用户读到的是“这个人是谁、做了什么、为什么值得被记住”。

5) 影视资源模块

影视模块的前台体验像“资源书架”：列表页展示海报卡片、支持按类型过滤；详情页展示简介，并提供“前往观看/了解”的按钮跳转到 `url`。列表的封面统一从 `getCoverUrl()` 取值：优先 `path`，再退到 `url`，仍为空则用占位图。这个兜底策略非常现实：它不是为了“优雅”，是为了确保演示时不会出现一排裂图。

后台影视资源目前为物理删除（`$model->delete()`），但从内容治理角度，未来可以对齐人物策略，增加 `deleted_at`，实现“下架但保留”。当你们把 `MediaResource` 更深度地挂载到人物/事件/战役上时，软删会比硬删更安全。

6) 数据可视化模块

数据可视化模块更像“史料索引”：它把抽象的伤亡、规模、损失等信息变成可读的图形。项目早期我们并不急着把每份 CSV 都拆成数据库明细表（成本高、字段也可能变），而是先用 `war_dataset` 把 CSV 管成“目录”。

实现逻辑是：

后台维护 `war_dataset` (`key/name/source_file/is_active/display_order`) → 前台先查询 `is_active=1` 的数据集列表并按 `display_order` 输出目录 → 用户点击某条数据集 → 前端根据 `source_file` 读取对应 CSV 并渲染表格/图表 → 图下标注数据来源或说明（可从 `description` 读取），避免“只剩图，没有出处”。

这样做的好处是：数据页面不再依赖写死文件名；数据集要上架/下架、排序、补说明，都能在后台完成，符合纪念站“能治理”的要求。

7) 留言模块

留言模块是站点的温度，但也天然带来治理压力。相比“提交后立刻展示”，本项目更倾向于“先收集、再审核、后展示”的稳妥路径：用户提交时获得明确提示，管理员在后台完成治理动作，前台只展示已通过内容，从而把风险控制在管理端。

(1) 模块交互流程

前台留言提交流程：

```
用户访问留言板 → 填写表单（昵称+内容） → 提交表单
↓
Controller 接收请求 → Model 验证数据 → 数据库保存（强制 is_approved=0）
↓
返回结果 → 刷新页面 → 前台列表仍只显示已审核留言
```

后台管理流程：

```
管理员登录 → 进入留言管理 → 查看留言列表（按时间倒序）
↓
查看详情 / 编辑内容 / 调整审核状态 / 删除
↓
执行 CRUD 操作 → 更新数据库 → 前台展示随审核状态变化
```

这种交互看起来“多了一步审核”，但对于纪念站是必要的：它让公开展示的内容始终保持可控，也让“纪念”不被噪声稀释。

(2) 接口设计 (路由约定)

前台接口：

路由	方法	功能	参数
/guestbook-message/index	GET/POST	显示留言板/提交留言	nickname, content

后台接口：

路由	方法	功能	参数
/guestbook-message/index	GET	留言列表	page, per-page
/guestbook-message/view	GET	查看详情	id
/guestbook-message/create	GET/POST	创建留言	nickname, content,is_approved
/guestbook-message/update	GET/POST	编辑留言	id, nickname, content, is_approved

说明：后台接口中出现 create，是为了支持管理员补录或代为整理留言（例如把线下纪念活动的寄语录入系统）。这类功能在纪念站并不多见，但它能让留言板不只是一块“即时评论区”，也可以沉淀更正式的纪念文字。

(3) 控制器设计 (前台提交 + 列表展示)

```
<?php
namespace frontend\controllers;

use Yii;
use yii\web\Controller;
use common\models\GuestbookMessage; // <-- 引入模型

class GuestbookController extends Controller
{
    public function actionIndex()
    {
        $model = new GuestbookMessage();

        if ($model->load(Yii::$app->request->post())) {
            $model->is_approved = 0; // 强制新留言为"未审核"
            if ($model->save()) {
                Yii::$app->session->setFlash('success', '您的留言已提交，请等待管理员审核。');
                return $this->refresh(); // 防止刷新重复提交
            }
        }

        $messages = GuestbookMessage::find()
            ->where(['is_approved' => 1]) // 只显示"已审核"的留言
            ->orderBy(['created_at' => SORT_DESC])
```

```
        ->all();

        return $this->render('index', [
            'messages' => $messages,
            'model' => $model,
        ]);
    }
}
```

在实现层面，这段逻辑体现了留言模块最重要的“治理原则”：**前台永远只展示已审核内容**。用户体验上也不会显得冷冰冰，因为提交后会得到明确反馈；内容治理上，管理员拥有最后的展示权。

4. 安全设计（输入、输出、会话三条线）

系统安全不需要“写得很吓人”，但要把常见风险堵住：

- 输入侧：ActiveRecord 规则校验、长度限制、必要字段必填
- 数据库侧：参数化查询（Yii 默认），避免拼 SQL
- 输出侧：视图编码输出，必要时用 HtmlPurifier 过滤富文本
- 表单侧：CSRF 默认开启
- 留言侧：验证码 + 审核状态，减少垃圾信息与恶意脚本

5. 风险点与可扩展方向

项目后续最值得做的增强，其实都围绕“内容治理”与“内容可信”：

- 时间轴的长文从 `detailMap` 入库（增加 `detail` 字段），让后台可编辑可审核
- `MediaResource` 增加软删，与 `Figure` 的治理策略一致
- 增加“来源/引用”字段（URL/书目），让人物、事件、数据都可追溯
- 图片上传从“填链接”升级为“真正上传”（存相对路径），彻底摆脱外链不稳定
- 搜索与筛选（人物按姓名/全文；影视按类型/年份；事件按关键词）
- 利用 `linkable_type/linkable_id` 在人物详情页推荐关联影视资源，形成内容互链
- 留言模块可扩展：增加管理员回复、敏感词标记、展示“精选寄语”、按时间归档等，让留言板从“评论区”成长为“纪念墙”

这些扩展不要求你们立刻做完，但只要现在的数据结构与路由约定合理，后续加功能就不会推倒重来。

6. 页面入口与路由汇总

前台常用入口（语义化路径）：

- `/site/index`（首页）
- `/timeline/index`、`/timeline/view?id=...`
- `/battle/index`、`/battle/view?id=...`
- `/figure/index`、`/figure/view?id=...`
- `/media/index`、`/media/view?id=...`

- `/message/index`
- `/guestbook-message/index` (留言板: 展示 + 提交)

后台常用入口 (标准 `r=` 路由) :

- `index.php?r=homepage-feature/index`
- `index.php?r=timeline-event/index`
- `index.php?r=battle/index`
- `index.php?r=figure/index`
- `index.php?r=media-resource/index`
- `index.php?r=guestbook-message/index` (留言管理: 审核/编辑/删除)